Frontier Models as Weight Editors for Gemma 4 E4B

A gradient-free weight edit cuts a coding agent's escape-code errors from 15 % to 3 % in live runs, and shows what such fixes must also preserve

Abstract

We study whether frontier models can change a coding agent's behaviour by editing its weights directly. The target is one defect of Gemma 4 E4B: when it edits a file, it often copies escape codes such as `\n` into the text it means to replace. Editors could read the weights and send at most 25 forward-pass requests, but could not compute gradients. In a test fixed in advance on 69 held-out repositories, three submitted edits cut these escape artefacts by 23, 53 and 75 %, beyond size-matched random controls, but each also lost 3 to 11 of 213 previously working edit calls in a draw, where our preservation rule allowed 2. Served as a LoRA adapter on 4-bit E4B in vLLM, one selected edit cut the defect in live agent runs on 148 new repositories from 15 % to 3 % of edit calls, and from 7 % to 1 % with thinking on, in two replications planned before their runs, and it ran in the competition's own scorer. Gradient training on the same calls removed more artefacts and failed the same rule. We detected no difference in solved tasks; the harness already repairs many escaped copies, limiting the expected benefit. We release the library, a new task set and the replay test set.

1. Introduction

Gemma 4 E4B is a small open model that works as a coding agent in the Gemma 4 Developer Agent Competition's harness. To change a file, the agent calls an edit tool with the *old text*, copied exactly from the file, and the new text. The model sees files in an escaped form, where a line break is the two characters `\n` and a quote is `\"`, and it often copies that form into the old text (Figure 1). We call each such leftover an *escape artefact*.

(a) The file on disk

```
def total(items):
    return sum(item["price"] for item in items)
```

(b) What the model reads: the same text inside the tool result, written with escape codes

```
{"content": "def total(items):\n    return sum(item[\"price\"] for item in items)\n"}
```

(c) The old text the model writes when it edits this file

Unedited model

```
def total(items):\n    return sum(item[\"price\"] for item in items)
```

✗ not in the file; only a fallback that removes escapes rescues it

Edited model

```
def total(items):
    return sum(item["price"] for item in items)
```

✓ found in the file as written

escape code: \n stands for a line break, \" for a quote

Figure 1. The defect, in a synthetic example: the unedited model copies escape codes into the old text; the edited model writes the real characters.

The harness already softens this defect. When an old text is not found, a fallback unescapes both the old text and its replacement and tries again; in our training runs it rescued 108 of 614 accepted edits. So we did not expect fewer escape artefacts to solve many more tasks. We chose the defect because it is frequent, countable and specific. That makes it a clean test of a narrower question: can a frontier model deliberately change one behaviour of a small model by writing its weights, without breaking the calls that already work?

Frontier models have recently been shown to write changes to neural-network weights directly, an approach called

weight-space programming [1]. We test the idea on one behaviour of a software-engineering agent, along three separate questions (Figure 2): does the behaviour change, are working calls preserved, and does the agent solve more tasks?

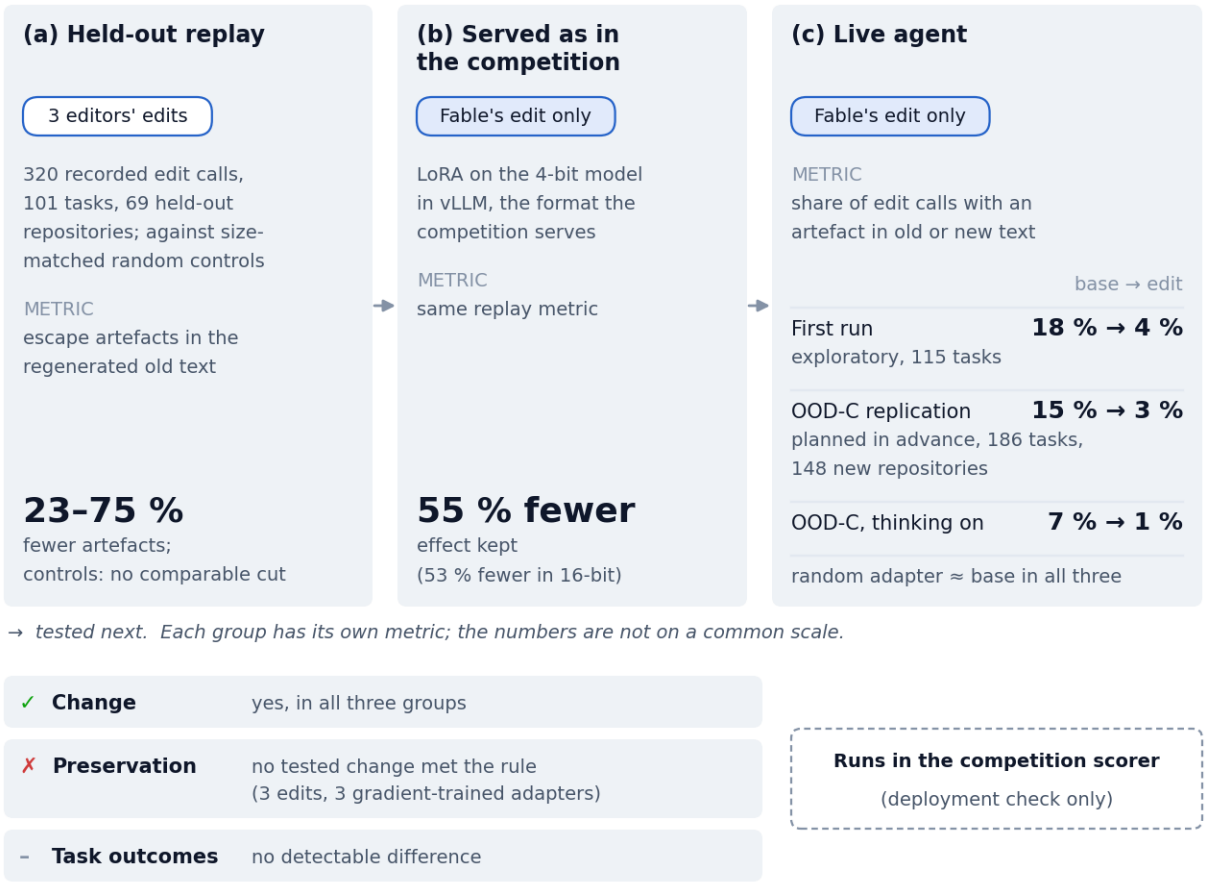


Figure 2. Evidence map: three groups of tests, each with its own metric and edits; arrows mean "tested next". Below: the three outcomes and a deployment check.

We contribute (1) a controlled experiment with frontier-model weight editors, held-out repositories and size-matched random controls; (2) tests of one selected edit in live, quantized serving and against gradient training on the same calls; and (3) a library, a task set and a replay set for repeating such tests, described in a companion writeup. The experiment tests particular edits under fixed budgets; it does not map the limits of weight editing.

2. Method

Each editor worked alone in a sandbox, with the weights and recorded calls from four training repositories but no access to the test data (Figure 3a). Claude Opus 5.5 and Claude Fable 5.1 each had 25 forward-pass requests and GPT-6 Astra had 10; a request could batch calls and return outputs or activations. The edits are *gradient-free*: backpropagation and iterative optimisers were not allowed, but closed-form fits on activations, such as least squares, were. A separately registered informed Opus run could read the earlier editors' work and had 10 requests. Two later Fable 5.2 runs tried an informed edit and the repair of a gradient-trained adapter.

Each submission is a set of low-rank changes: a weight matrix W becomes W + BA, where B and A are two thin matrices. This form is a LoRA adapter [2], which the competition's vLLM server can load. Fable's edit, for example, is a rank-2 change in the last layer's feed-forward output. One part detects activations just before an escaped punctuation token, and the other shifts token scores toward the raw form; both are fitted in closed form. Opus instead turned about 40 little-used feed-forward neurons into detectors.

We judged each frozen edit by replay (Figure 3b). For a recorded edit call, the model regenerates only the old text from the exact original prompt, and the harness's matching code checks it against the file. On a fidelity subset, the

replay reproduced 95 of 103 recorded failures and all 213 successes. The test was fixed in advance [3]: 320 calls from 101 tasks in 69 held-out repositories, each generated in two *draws*, one greedy and one at temperature 0.6. Each edit faced the base model and two random controls with the same location, rank and size but a random direction, and its per-task artefact rate had to fall against all three (one-sided sign tests, Holm-corrected across candidates). Three guards limited side effects:

- acceptance, the share of old texts found, averaged over tasks, could fall by at most 1 point;
- among recorded calls that had succeeded and that the base model's replay also matched (213 per draw), which we call *working calls*, at most 2 could be lost in each draw; these are accepted old-text matches, not correct bug fixes;
- compile failures among accepted calls could rise by at most 2 points, measured on a replay that regenerates both the old and the new text, because the fallback changes both. This full-call form replaced the registered one before any edit ran on the test calls; the full paper gives both verdicts.

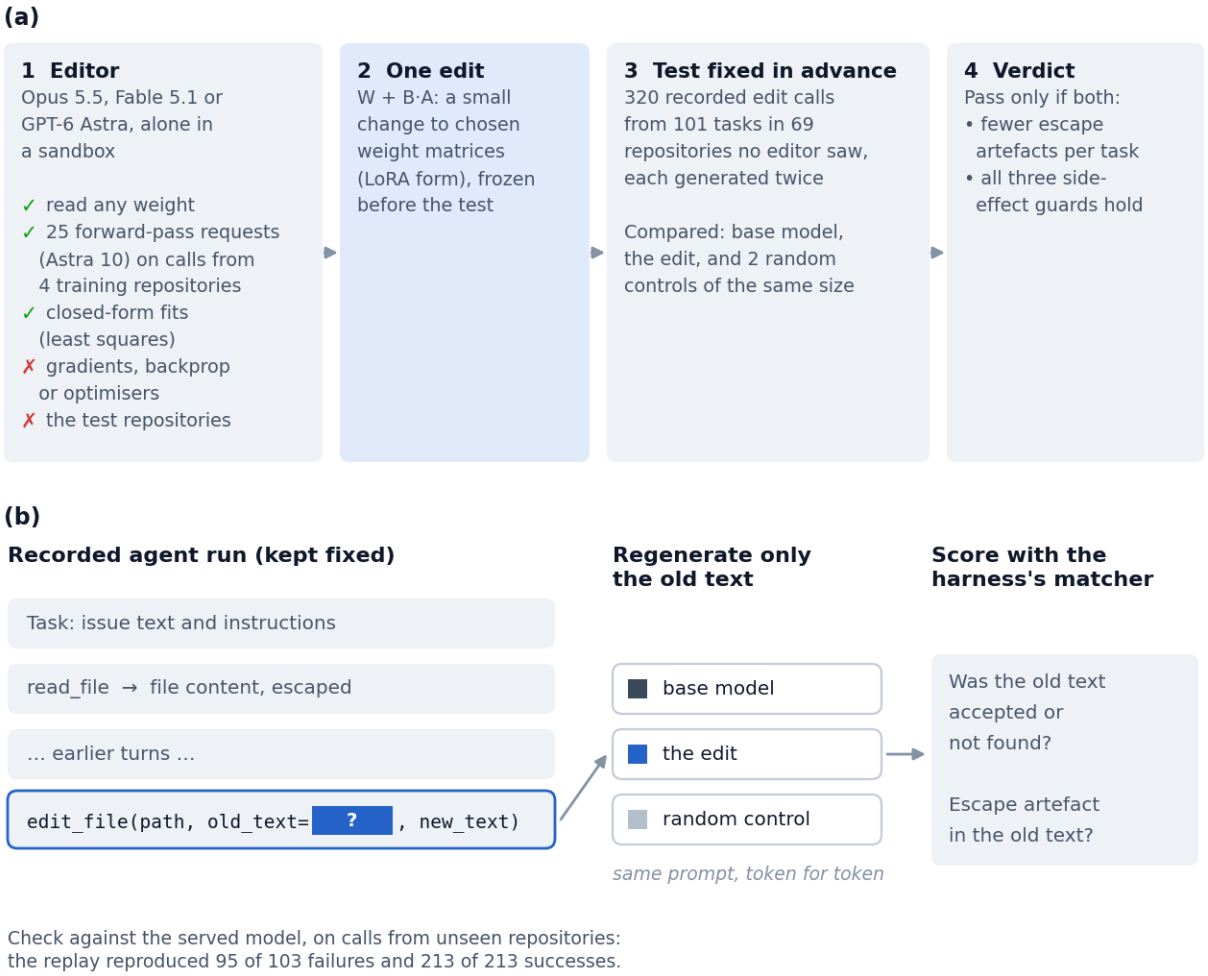


Figure 3. (a) What an editor could and could not do. (b) The replay regenerates only the old text; everything before it stays fixed.

The replay calls come from agent runs on two held-out sets, OOD-43 and OOD-B, and the first live run used their 115 tasks. For the planned replications we built OOD-C: 186 tasks from 148 repositories, filtered from SWE-rebench [4, 5], packaged to run offline in the competition harness, and checked by script to share no repository with training or the earlier sets. Only 53 OOD-C tasks meet our full gradability rule: the reference fix passes twice, the empty patch fails, and the reference fix passes again under Python 3.12. So OOD-C mainly measures what the agent writes. In live runs we count edit calls whose old *or* new text carries an artefact, unlike the old-text replay, and leave out calls whose file state could not be rebuilt.

3. Results

3.1 Targeted change and preservation

All three second-round edits reduced the defect, and every one lost more working calls than the rule allowed (Figures 4 and 5a). Of 640 replay generations, 100 of the base model's carried an artefact; Opus's edit left 77, Fable's 47 and informed Opus's 25: cuts of 23, 53 and 75 %, each with $p \leq 0.011$ against the base and both of its controls. The six random controls left 99 to 102. But the three edits lost up to 3, 5 and 11 of the 213 working calls in a draw, against a limit of 2, and Fable's (narrowly, −1.01 points) and informed Opus's edits also failed the acceptance guard.

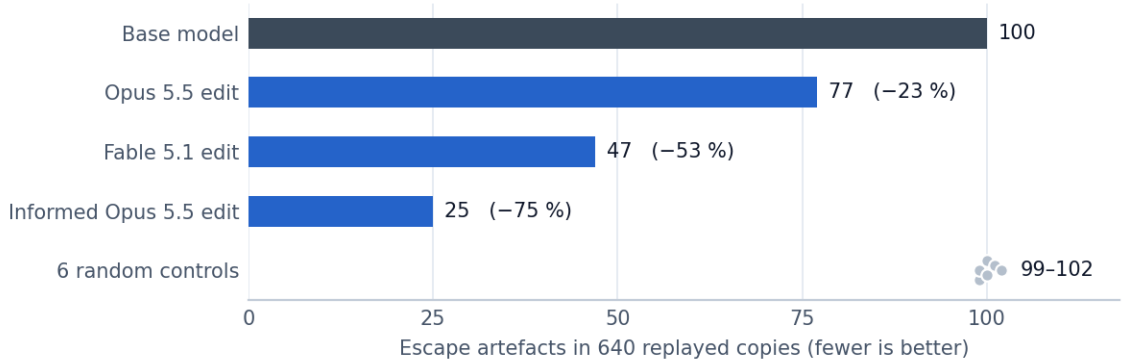


Figure 4. Every edit cut escape artefacts beyond its own size-matched random controls; the edits differ in size.

A cleaner-looking copy can be less reliable than an escaped one. When the base model wrote an escaped old text, the fallback often repaired it and the call succeeded. Writing raw text there gains nothing, and the raw copy is sometimes wrong, for example with a closing brace dropped (Figure 5b). Of the escaped copies that the fallback had rescued, the three edits moved 24, 45 and 67 off the fallback, and 5, 12 and 24 of those failed. Average acceptance can hide such losses when other calls improve, so the rule counts them separately.

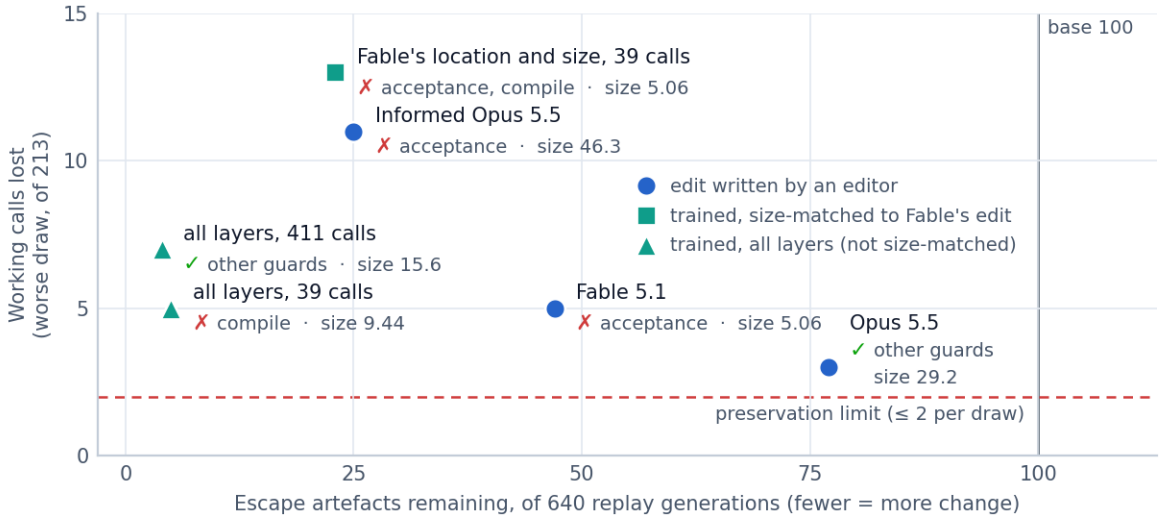
The editors' decisions add context. Informed Opus named the lost-call guard as its main risk and submitted anyway. Astra submitted nothing; its best candidate showed no effect on its set-aside tasks. Both Fable 5.2 runs found effects but withdrew their candidates after measuring side effects themselves. These are single runs, not a ranking of models. Gated afterwards off the record, one withdrawn candidate showed why it was withdrawn: 43 artefacts against 100, but 3 working calls lost per draw and acceptance 1.27 points lower.

3.2 Comparison with gradient training

Editing by reasoning would be most useful if it needed less data than training. In a comparison fixed in advance, we trained LoRA adapters with gradients to write the correct old text on calls from the training repositories. The matched arm used the 39 labelled calls among those Fable had read, at Fable's location, rank and size. Two larger arms trained all linear layers, on the same 39 calls and on all 411 labelled calls; these two are not size-matched. Only the 39-call arms tuned their learning rate, on validation calls from the training repositories.

Training removed more artefacts and failed the same rule (Figure 5a). The three arms left 23, 5 and 4 artefacts against Fable's 47; at Fable's location the trained adapter was lower in 13 tasks and higher in 1 ($p = 0.002$). They lost 13, 5 and 7 working calls in their worse draw. The 411-call arm passed the acceptance and compile guards and still exceeded the limit.

(a) Behaviour change and preservation, six candidates



✓ ✗: the other guards, acceptance and compile (see the full paper). Size: Frobenius norm of the weight change.

(b) How a raw copy breaks (synthetic example)

File	`return {"total": total(items), "count": len(items)}`
Unedited model	`return {\"total\": total(items), \"count\": len(items)}` ✓ escaped, rescued by the fallback, applied
Edited model	`return {"total": total(items), "count": len(items)` ✗ raw, but the closing brace is missing: not found

Figure 5. (a) Artefacts left against working calls lost in the worse draw, for three edits and three gradient-trained adapters; all exceed the limit, and only the square is size-matched to Fable's edit. (b) How a raw copy breaks.

On this target the authored edit showed no data advantage over training, and the loss of working calls was not specific to gradient-free editing. This does not show that removing escape artefacts must break calls: the all-layer arms removed more while losing fewer.

3.3 Transfer to the running agent

We chose Fable's edit after seeing the replay results, so this step is exploratory. Served as a LoRA adapter on 4-bit (W4A16) E4B in vLLM, it kept its effect: artefacts fell from 102 to 46, close to its 16-bit result, while a random adapter left 100. So the change survives quantized serving, and so does its cost: 7 and 11 working calls lost per draw (5 and 5 in 16-bit; none for the random adapter). Agents with this edit also ran in the competition's scorer without errors, scoring 0.05 with thinking off and on, like our comparable agents without it (0.05 to 0.08).

In live agent runs the edit cut the defect again, on new repositories, under a plan fixed in advance (Figure 6). A first, exploratory run on the 115 OOD-43 and OOD-B tasks gave 4 % of counted edit calls with an artefact, against 18 % for the base and 20 % for a random adapter. Because those repositories had supplied the replay calls, we froze the edit, the analysis, the pass rule and an expected effect, and repeated the three arms on OOD-C. Artefacts fell from 15 % of counted edit calls for the base to 3 % for the edit, with 16 % for the random adapter. Per task, the edit was lower than the base in 34 tasks and higher in 4 ($p = 3 \times 10^{-7}$), and lower than the random adapter in 27 and higher in 7 ($p = 4 \times 10^{-4}$); a repository-level bootstrap also supported the cut. The live gain does not lift the replay's preservation failure.

The agent we submit thinks before it acts; the runs above did not. A second planned replication on the same OOD-C tasks, with thinking on, gave 7 % for the base, 1 % for the edit and 8 % for the random adapter. The edit was lower than the base in 16 tasks and higher in 4 ($p = 0.006$), and passed against the random adapter too ($p = 2 \times 10^{-4}$). Counting

only the old text gives the same picture in every run. The two replications ran in separate sessions, so their base rates do not measure what thinking does.

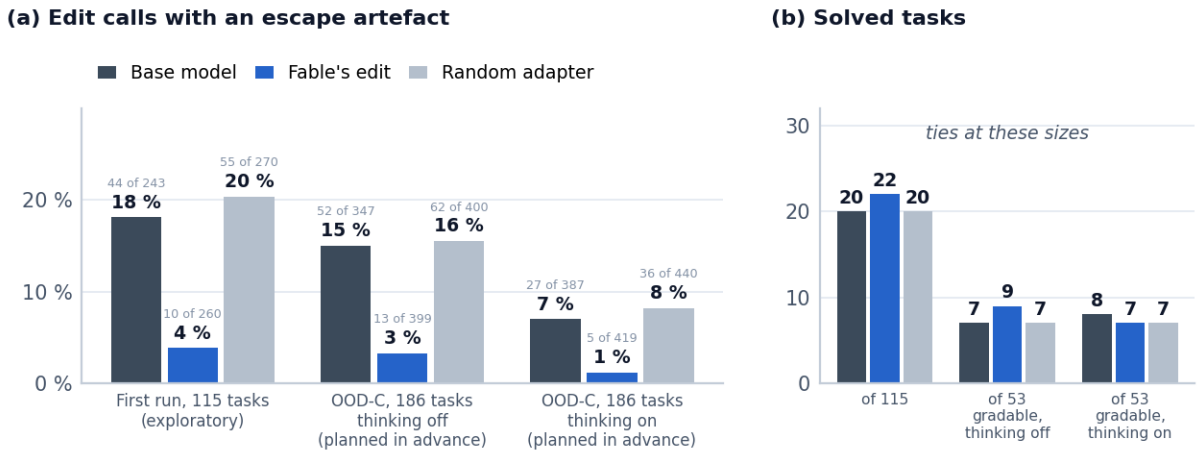


Figure 6. In live runs the selected edit cut escape artefacts with thinking off and on, and no difference in solved tasks was detected. Shares pool counted calls; the tests compare paired tasks.

3.4 Task outcomes

We detected no difference in solved tasks. For the base, the edit and the random adapter, the first run solved 20, 22 and 20 of 115 tasks; on the 53 gradable OOD-C tasks they solved 7, 9 and 7 with thinking off, and 8, 7 and 7 with thinking on. Comparisons this small cannot show equivalence. On OOD-C, calls whose old text was not found rose with both adapters (thinking off: 27 % base, 34 % edit, 32 % random adapter); its cause is untested.

Most of the base model's misses followed an earlier miss in the same task (131 of 202). Two checks, run in turn before a third round aimed at such retries, failed their rules, so we did not run it.

4. Related work and scope

Weight-space programming showed that frontier models can write changes to network weights directly, on small networks and a 0.8-billion-parameter model [1]; we add a deployed coding agent, held-out repositories, live runs and a gradient baseline. ROME writes a human-designed rank-one update where a fact is stored [6], and activation steering changes intermediate activations at inference time [7]. Left to themselves, the editors rebuilt known families of edits, but none tried updates that restrict or penalise changes on preserved calls' activations, such as a null-space constraint [9] or a covariance-weighted key-value edit [10]; with preservation as the binding constraint, these are the obvious next test, though they bound activations, not outputs. Agent success also depends on its tool interface [8]. In round 1, aimed at whether the old text is found at all, three weight edits gained nothing, while a tool-side change, showing tool results unescaped, raised acceptance by 2.0 points on the same calls ($p = 0.031$). We ran no steering or prompt comparison.

The study covers one model, one main target, one run per editor, one live run per arm in each setting, and one training seed. We chose the target after counting artefacts in the base model's test calls, and the deployed edit after the replay; the replications fix the edit, not those choices. Missing traces and file states reduce the paired samples, and no zero-weight adapter separates serving effects from perturbation effects. The round-2 rule was frozen locally before the first forward pass and published while the search was under way; the live-replication plans were logged before their runs and committed afterwards.

The lesson that carries forward: where a tool already repairs a malformed argument, removing the malformation cannot gain those calls and can lose some, so preservation must be tested apart from the target change, and live behaviour apart from both. Whether the approach helps another behaviour or model, or an independent user, remains open.

5. Resources

`weightprog` (Apache-2.0, alpha) packages edit constructions, model instruments, size-matched random controls and comparisons, with MLX and PyTorch backends. A companion writeup describes it and the data below. On the real model its base generation, backend agreement and captured edit effects match the study's outputs; edited generation has run through its MLX backend, and this study's adapters came from its export helper. A separate study kit holds this paper's replay, gate and editor sandbox.

OOD-C packages 186 tasks from 148 repositories for offline agent runs, built from SWE-rebench sources (CC-BY-4.0) with open-source licences only and with gradability flags; with snapshots and wheels it is 14.4 GB. The replay set holds the 320 held-out calls with their prompts and file states. The full paper adds round 1 and the retry checks, and the repository holds the protocols, records and audits. Library: github.com/ozgung/weightprog; OOD-C: kaggle.com/datasets/ozgung/g4-oodc; replay set: kaggle.com/datasets/ozgung/g4-weight-edit-replay; repository: github.com/ozgung/gemma4-weight-edit-study.

6. Conclusion

Frontier models wrote gradient-free weight edits that reduced a specific coding-agent behaviour beyond matched controls, and one edit kept that change in the live agent, with thinking off and on. No edit or gradient-trained adapter met our preservation rule, and no task benefit was detected: a targeted change with a measured cost, not an unavoidable trade-off.

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
3. van Miltenburg, van der Lee, Krahmer. Preregistering NLP Research. NAACL 2021.
4. Badertdinov et al. SWE-rebench. NeurIPS 2025.
5. Badertdinov et al. SWE-rebench V2. 2026.
6. Meng et al. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
7. Turner et al. Steering Language Models With Activation Engineering. arXiv 2023.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
10. Meng et al. Mass-Editing Memory in a Transformer. ICLR 2023.